Published in Towards Data Science

Satish Chandra Gupta Follow
Sep 28 · 10 min read · · Listen

Photo by Suzanne D. Williams on Unsplash

MLOPS

MLOps: Machine Learning Lifecycle

Machine Learning Lifecycle for MLOps era brings model and software development together to build ML-assisted products

- **Model Development:** Data Scientists — highly skilled in statistics, linear algebra, and calculus — train, evaluate, and select the best-performing statistical or neural network model.
- **Model Deployment:** Developers — highly skilled in software design and engineering — build a robust software system, deploy it on the cloud, and scale it to serve a huge number of concurrent model inference requests.

Of course, that is a gross over-simplification. It takes several other vital expertise in building useful and successful ML-assisted products:

- **Data Engineering:** Build data pipelines to collect data from disparate sources, curate and transform it, and turn it into homogenous, clean data that can be safely used for training models.
- **Product Design:** Understand business needs, identify impactful objectives and relevant business matrices; define product features or user stories for those objectives, recognize the underlying problems that ML is better suitable to solve; design user experience to not only utilize ML model prediction seamlessly with rest of the product features but also collect user (re)action as implicit evaluation of the model results, and use it to improve the models.
- **Security Analysis:** Ensure that the software system, data, and model are secure, and no Personally Identifiable Information (PII) is revealed by combining model results and other publicly available information or data.
- **AI Ethics:** Ensure adherence to all applicable laws, and add measures to protect against any kind of bias (e.g. limit the scope of the model, add human oversight, etc.)

As more models are being deployed in production, the importance of MLOps has naturally grown. There is an increasing focus on the seamless design and functioning of ML models within the overall product. Model Development can't be done in a silo



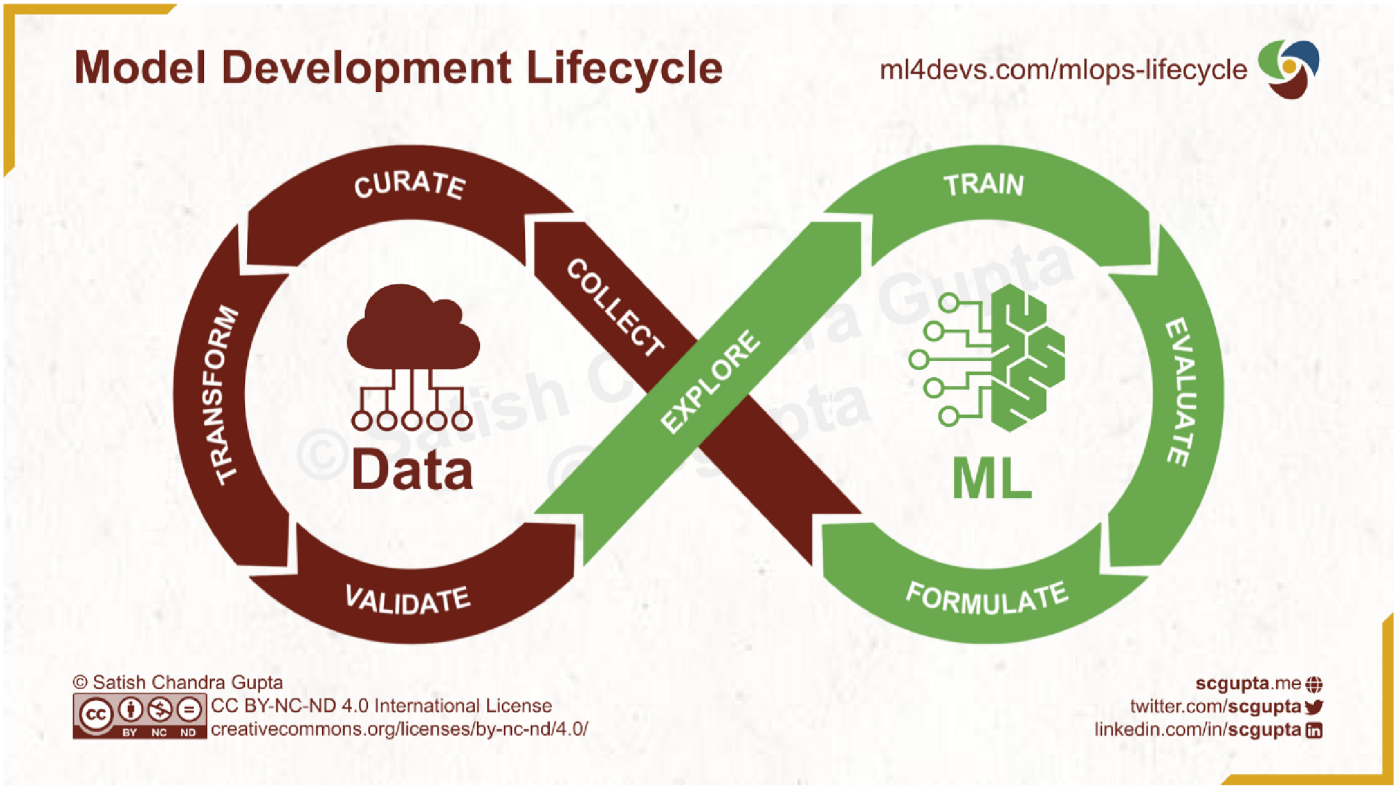

changes in the existing workflows of data scientists and engineers.

In the rest of the article, I first give an overview of the typical Model Development and Software Development workflows, and then how to bring the two together for adapting to the needs of building ML-assisted products in the MLOps era.

Model Development Lifecycle

Let's set aside deploying models online into production for a moment. Data Scientists have been building statistical and neural-net models for over a decade. Often, these models were used offline (i.e. executed manually) for predictive analytics.

Model development consists of two sets of activities: data preparation and model training. It starts with formulating an ML problem and ends with model evaluations.



Model Development Lifecycle. Image by the author.

- **Business Objective:** Narrow down to a small set of ML problems that can serve the business objective.
- **Cost of Mistakes:** No ML model can be 100% accurate. What are the cost of false positives and false negatives? For example, if an image classification model wrongly predicts breast cancer in a healthy person, further tests will rectify it. But if the model fails to diagnose cancer in a patient, then it can turn out to be fatal due to late detection.
- **Data Availability:** It may come as a surprise, but you may start with no data and bootstrap your data collection. As the data becomes richer, it may make more types of models viable. For example, if you were to do anomaly detection with no labeled data, you may start with various kinds of unsupervised clustering algorithms and mark points that are not in any cluster as anomalies. But as you collect user reactions to your model, you will have a labeled dataset. Then you may want to try if a supervised classification model will perform better.
- **Evaluation Metrics:** Depending upon problem formulation, you also should specify a model performance metric to optimize for, which should align with the business metric for your business objective.

Collect

Collect the necessary data from internal applications as well as external sources. It may be by scrapping the web, capturing event streams from your mobile app or web service, Change Data Capture (CDC) streams from operational (OLTP) databases, application logs, etc. You may ingest all of the needed data into your data pipeline, which is designed and maintained by Data Engineers.

Curate

Collected data is almost never pristine. You need clean it, remove duplicates, fill in missing values, and store it in a data warehouse or a data lake. If it is for training a supervised ML model, then you will also have to label it. Also, you must catalog it so




Transform

Once data has been cleaned, you can transform it to suit the analytics and ML modeling. It may require changing the structure, joining with other tables, aggregating or summarizing along important dimensions, computing additional features, etc. Data Engineers should automate all of it in the data pipeline.

Validate

Implement quality checks, maintain logs of statistical distributions over time, and create triggers to alert when any of the checks fail or the distribution sways beyond expected limits. Data Engineers in consultation with Data Scientists implement these validations in the data pipeline.

Explore

Data Scientists perform Exploratory Data Analysis (EDA) to understand the relationships between various features and the target value they want the model to predict. They also do Feature Engineering, which is likely to lead to adding more transformation and validation checks (the previous two stages).

Train

Data Scientists train multiple modes, run experiments, compare model performance, tune hyper-parameters, and select a couple of best-performing models.

Evaluate

Evaluate the model characteristics against business objectives and metrics. Some feedback may result in even tweaking and formulating the ML problem differently, and repeating the whole process all over again.

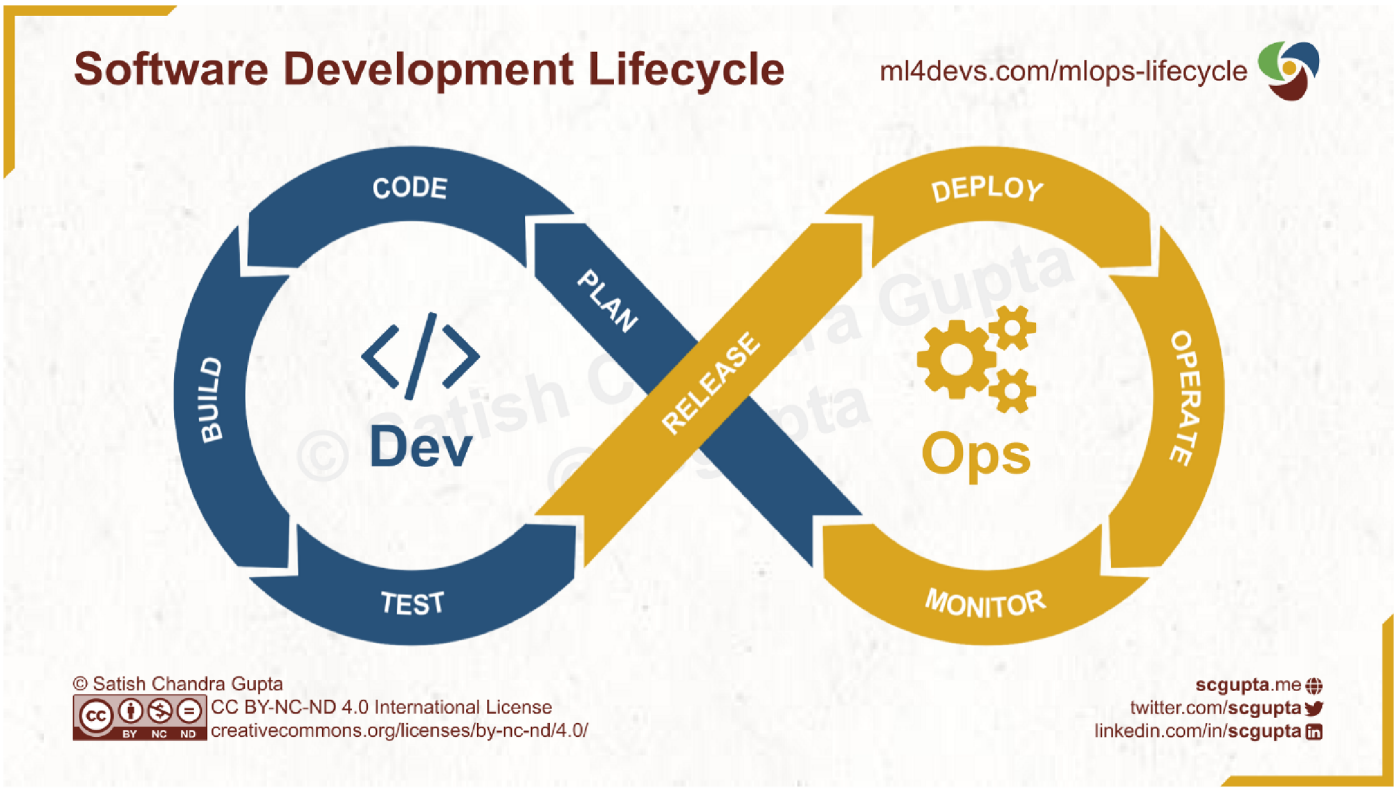
This Data-ML infinite loop is not linear. At every stage, you don't always move forward to the next stage. Upon discovering problems, you go back to the relevant previous stage to fit them. So there are implicit edges from each stage to previous stages.




Software Development Lifecycle

The DevOps infinite loop is the de-facto standard for the software development lifecycle to rapidly build and deploy software applications and services on the cloud.

It consists of two sets of activities: designing and developing a software system, and deploying and monitoring software services and applications.



Software Development Lifecycle. Image by the author.

Plan

This is the first stage for any product or product feature. You discuss the business objectives and key business metrics, and what product features can help achieve them. You drill down into the end-user problems and debate about user journeys to address those problems and collect required data to assess how an ML model is performing in the real world.

inference and consume its results, and also what user reactions and feedback will be collected.

It is very important to get developers, data engineers, and data scientists to get here on the same page. That will reduce nasty surprises later.

Build

This stage fuels the Continuous Integration of various parts as they evolve and package into a form that will be released. It can be a library or SDK, a docker image, or an application binary (e.g. apk for Android apps).

Test

Unit tests, integration tests, coverage tests, performance tests, load tests, privacy tests, security tests, and bias tests. Think of all kinds of software and ML mode tests that are applicable here and automate them as much as feasible.

Testing is done on a staging environment that is similar to the targeted production environment but not designed for a similar scale. It may have dummy, artificial, or anonymized data to test the software system end-to-end.

Release

Once all automated tests pass and, in some cases, test results are manually inspected, the software code or models are approved for release. Just like code, models should also be versioned and necessary metadata automatically captured. Just as the docker images are versioned in a docker repo, the model should also be persisted in a model repo.

If models are packaged along with the code for the microservice that serves the model, then the docker image has the model image too. This is where Continuous Integration ends and Continuous Deployment takes over.

Deploy

Picking the released artifacts from the docker repo or model store and deploying it on




You may also use TensorFlow Serve, PyTorch Serve, or services like SageMaker and Vertex AI to deploy your model services.

Operate

Once the services are deployed, you may decide to send a small percentage of the traffic first. Canary Deployment is common tactic to update in phases (e.g. 2%, 5%, 10%, 25%, 75%, 100%). In case of a problem, unexpected behavior, or a drop in metrics, you can roll back the deployment.

Once the gate is opened to 100% traffic, your deployment infra should gracefully bring down the old service. It should also scale as the load peaks and falls. Kubernetes and KubeFlow are common tools for this purpose.

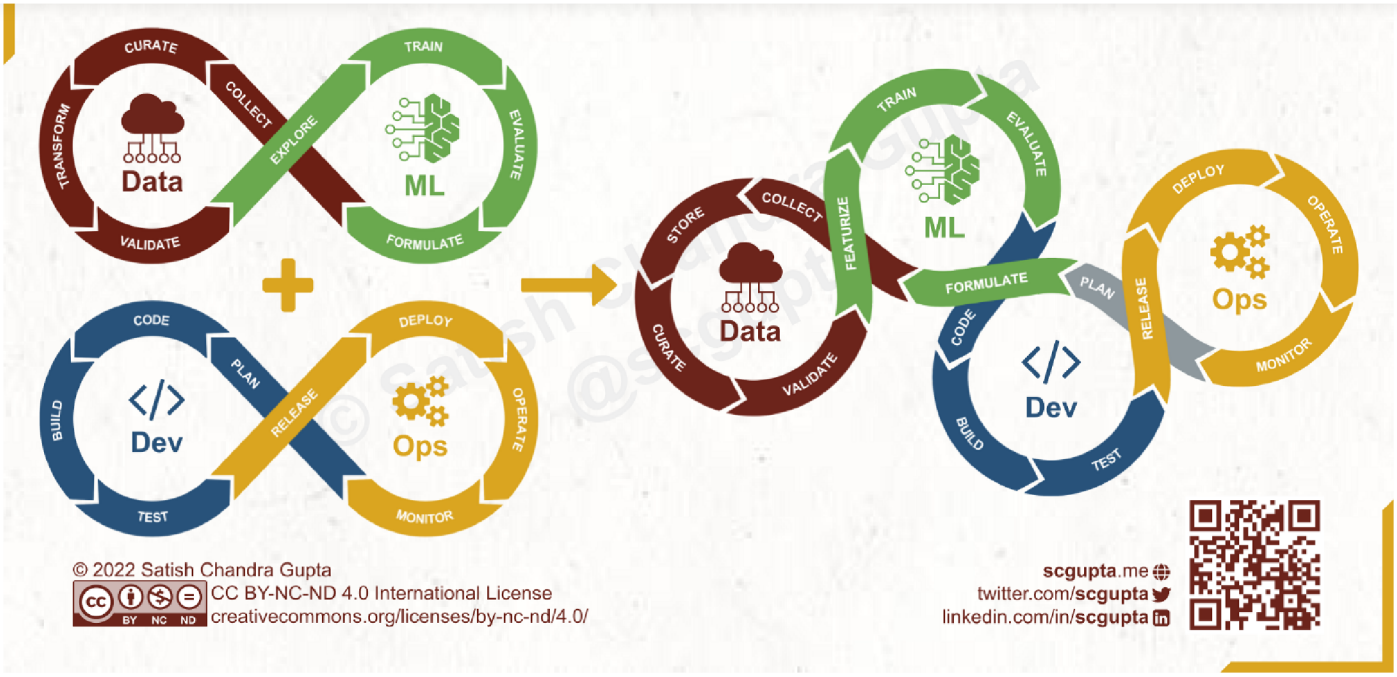
Monitor

In this final phase, you constantly monitor the health of services, errors, latencies, model predictions, outliers and distribution of input model features, etc. In case a problem arises, depending upon the severity and diagnosis, you may roll back the system to an older version, release a hotfix, trigger model re-training, or do whatever else is needed.

ML Lifecycle in MLOps Era

At the moment, it is quite common for data scientists to develop a model and then "throw it over the wall" to developers and ML engineers to integrate with the rest of the system and deploy it in production.

The ML and Dev silos and fragmented ownership are one of the most common reasons why many ML Projects fail. Unifying the model and software development lifecycles provides much-needed visibility to all stakeholders.



Model Development and Software Development need to stitch together for building ML-assisted products. Image by the author.

Plan Step is the Starting Point

Product planning comes before everything else. Defining business objectives and designing user experiences should include not just product functionality, but also how model results and capturing user reactions will be blended into the production design.

Unlike traditional software, when more data is collected with time, the user experience of the ML aspects of a product may need an update to benefit from it, even though there is no “new functionality.”

First Build the Product Without ML

I often first build an end-to-end application with a rule-based heuristics or dummy model, cutting off the Data-ML loop entirely. That works as a baseline model and is useful in collecting data. It also gives context to the data scientists by showing how the model will be used in the product.

Different Cadence for Model and Software Development

Developing an ML model is quite different from developing software. Software systems

A single lifecycle does not preclude Data, ML, Dev, and Ops wheels spinning at different speeds. In fact, it already happens in DevOps. At some teams, not every Dev sprint results in a new version being deployed. On the other hand, some teams deploy new versions every hour, i.e. hundreds of times in a single sprint. Let every wheel spin at its own optimal speed.

Consolidated Ownership, Integrate Early, Iterate Often

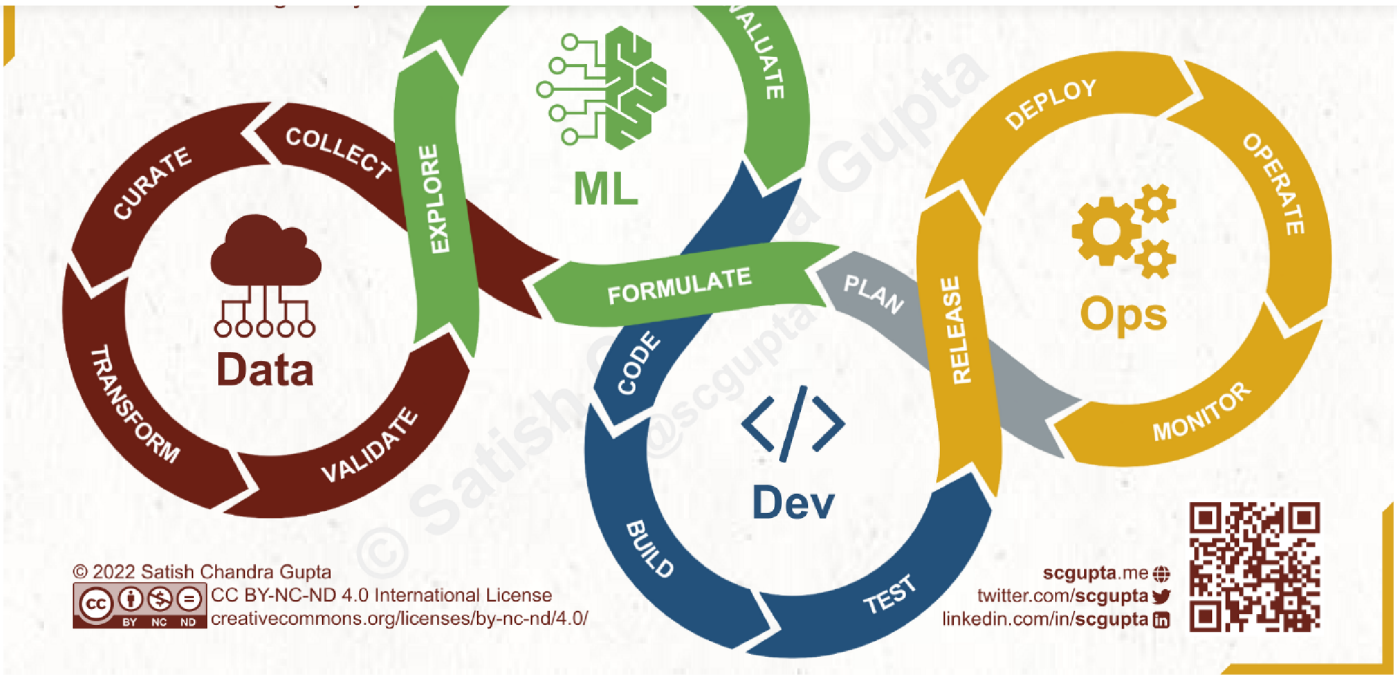
These are my 3 percepts for improving the success rate in developing and deploying ML-assisted products:

- **Consolidate Ownership:** Cross-functional team responsible for the end-to-end project.
- **Integrate Early:** Implement a simple (rule-based or dummy) model and develop a product feature end-to-end first.
- **Iterate Often:** Build better models and replace the simple model, monitor, and repeat.

Summary

Machine Learning Lifecycle for MLOps era brings model development and software development together into one eternal knot. It facilitates visibility to all stakeholders in building ML-assisted products and features.



Machine Learning Lifecycle in the MLOps era integrates Data, ML, Dev, and Ops in a single loop. Image by the author.

If you enjoyed this, please:

Originally published on ML4Devs.com.